```
#ifndef LIST_H
#define LIST_H
/* List.h
 *
 * doubly-linked, double-ended list with Iterator interface
 * Project UID c1f28c309e55405daf00c565d57ff9ad
 * EECS 280 Project 4
 */

#include <iostream>
#include <cassert> //assert
#include <cstddef> //NULL


template <typename T>
class List {
  //OVERVIEW: a doubly-linked, double-ended list with Iterator interface
public:
  List() : first(nullptr), last(nullptr){}

  List(const List &other)
   : first(nullptr), last(nullptr) {
     copy_all(other);
   }

  ~List(){
    clear();
  }

  List & operator=(const List &rhs){
    if (this == &rhs) {return *this;}
    clear();
    copy_all(rhs);
    return *this;
  }

  //EFFECTS:  returns true if the list is empty
  bool empty() const{
    if(first == NULL){return true;}
    return false;
  }

  //EFFECTS: returns the number of elements in this List
  int size() const{
    int size = 0;
    Node *ptr = first;
    while(ptr){
      size++;
      ptr = ptr->next;
    }
    return size;
  }

  //REQUIRES: list is not empty
  //EFFECTS: Returns the first element in the list by reference
  T & front() {
    assert(!empty());
    return first->datum;
  }
```

```
//REQUIRES: list is not empty
//EFFECTS: Returns the last element in the list by reference
T & back(){
  assert(!empty());
  return last->datum;
}

//EFFECTS:  inserts datum into the front of the list
void push_front(const T &datum){
  Node *p = new Node;
  p->datum = datum;
  p->prev = NULL;
  p->next = first;
  if(first == NULL){
    last = p;
  }
  else{
    first->prev = p;
  }
  first = p;
}

//EFFECTS:  inserts datum into the back of the list
void push_back(const T &datum){
  Node *p = new Node;
  p->datum = datum;
  p->prev = last;
  p->next = NULL;

  if(last == NULL){
    first = p;
  }
  else{
    last->next = p;
  }
  last = p;
}

//REQUIRES: list is not empty
//MODIFIES: may invalidate list iterators
//EFFECTS:  removes the item at the front of the list
void pop_front(){
  assert(!empty());
  if(first == last){
    Node *n = new Node;
    n = first;
    first = NULL;
    last = NULL;
    delete n;
  }
  else{
    Node *n = new Node;
    n = first;
    first = first->next;
    first->prev = NULL;
    delete n;
  }
}
```

```
  //REQUIRES: list is not empty
  //MODIFIES: may invalidate list iterators
  //EFFECTS:  removes the item at the back of the list
  void pop_back(){
    assert(!empty());
    if (first==last){
      Node *n = new Node;
      n = first;
      first = NULL;
      last = NULL;
      delete n;
    }
    else{
      Node *n = new Node;
      n = last;
      last = last->prev;
      last->next = NULL;
      delete n;
    }
  }

  //MODIFIES: may invalidate list iterators
  //EFFECTS:  removes all items from the list
  void clear(){
    while (!empty()){
      pop_front();
    }
  }

  // You should add in a default constructor, destructor, copy constructor,
  // and overloaded assignment operator, if appropriate. If these operations
  // will work correctly without defining these, you can omit them. A user
  // of the class must be able to create, copy, assign, and destroy Lists

private:
  //a private type
  struct Node {
    Node *next;
    Node *prev;
    T datum;
  };

  //REQUIRES: list is empty
  //EFFECTS:  copies all nodes from other to this
  void copy_all(const List<T> &other){
    for (Node *np = other.first; np; np = np->next){
      push_back(np->datum);
    }
  }

  Node *first;   // points to first Node in list, or nullptr if list is empty
  Node *last;    // points to last Node in list, or nullptr if list is empty

public:
  ////////////////////////////////////
  class Iterator {

    //OVERVIEW: Iterator interface to List
```

```
  // You should add in a default constructor, destructor, copy constructor,
  // and overloaded assignment operator, if appropriate. If these operations
  // will work correctly without defining these, you can omit them. A user
  // of the class must be able to create, copy, assign, and destroy Iterators.

  // Your iterator should implement the following public operators: *,
  // ++ (prefix), default constructor, == and !=.

public:
  // This operator will be used to test your code. Do not modify it.
  // Requires that the current element is dereferenceable.
  Iterator ( ) : node_ptr(nullptr){ }

  T &operator*() const{
    assert(node_ptr != nullptr);
    return node_ptr->datum;
  }

  Iterator& operator++() {
    assert(node_ptr);
    node_ptr = node_ptr->next;
    return *this;
  }

  bool operator==(Iterator rhs) const{
    return node_ptr == rhs.node_ptr;
  }

  bool operator!=(Iterator rhs) const{
    return node_ptr != rhs.node_ptr;
  }

  Iterator& operator--() {
    assert(node_ptr);
    node_ptr = node_ptr->prev;
    return *this;
  }

private:
  Node *node_ptr; //current Iterator position is a List node
  // add any additional necessary member variables here

  // add any friend declarations here

  // construct an Iterator at a specific position
  Iterator(Node *p) : node_ptr(p) {}
  friend class List;
};//List::Iterator
////////////////////////////////////

// return an Iterator pointing to the first element
Iterator begin() const {
  return Iterator(first);
}

// return an Iterator pointing to "past the end"
Iterator end() const{ return Iterator( );}
```

```
  //REQUIRES: i is a valid, dereferenceable iterator associated with this list
  //MODIFIES: may invalidate other list iterators
  //EFFECTS: Removes a single element from the list container
  void erase(Iterator i){
    if (i.node_ptr == first){
      first = i.node_ptr->next;
    }
    if(i.node_ptr->next != NULL){
      i.node_ptr->next->prev = i.node_ptr->prev;
    }
    if(i.node_ptr->prev != NULL){
      i.node_ptr->prev->next = i.node_ptr->next;
    }
    delete i.node_ptr;
  }

  //REQUIRES: i is a valid iterator associated with this list
  //EFFECTS: inserts datum before the element at the specified position.
  void insert(Iterator i, const T &datum){
    if(i.node_ptr != NULL){
      Node *newnode = new Node;
      newnode->datum = datum;
      newnode->prev = i.node_ptr->prev;
      i.node_ptr->prev = newnode;
      newnode->next = i.node_ptr;
      if(newnode->prev != NULL){
        newnode->prev->next = newnode;
      }
      else{
        first = newnode;
      }
    }

  }

};//List


////////////////////////////////////////////////////////////////////////
// Add your member function implementations below or in the class above
// (your choice). Do not change the public interface of List, although you
// may add the Big Three if needed.  Do add the public member functions for
// Iterator.


#endif // Do not remove this. Write all your code above this line.
```